

I am working on a Software Engineering lab assignment based on a Pygame Space Invaders project.

Repository:

https://github.com/SETAPESU26/46_space-invaders

I have forked the repository to my own GitHub account and cloned it locally.

The assignment requires me to improve the existing Space Invaders game with these four features:

1. Refine/fix bullet-enemy collision detection.
2. Add a proper graphical Game Over screen instead of only printing Game Over in the terminal.
3. Add replay functionality and Easy/Medium/Hard difficulty selection.
4. Add sound effects for:
 - player firing
 - enemy destroyed
 - game over

I want to implement these features properly in the existing codebase rather than rewriting the entire project.

Please work with me step-by-step:

- First inspect/explain the existing project structure and relevant files.
- Identify what needs to be changed for each requirement.
- Implement one requirement at a time.
- After each feature, explain what changed and how I should test it.
- Keep the existing game architecture and coding style where possible.
- Do not make unrelated changes.
- Don't skip directly to the final solution; guide me through the implementation and testing.

Start by examining the existing repository/code and tell me what

the current implementation does and what the first required change should be.

I inspected the actual repository and the current implementation. The good news is that the project is intentionally small, and **Task 1 has a very specific bug already documented in the code.**

Current project structure

The repository currently has:

</> Plain text

```
46_space-invaders/  
├─ main.py  
├─ requirements.txt  
├─ README.md  
└─ game/  
    ├─ game_engine.py  
    ├─ player.py  
    ├─ enemy.py  
    └─ bullet.py
```

The README confirms the four tasks you listed and explicitly says Task 1 is about bullets sometimes passing through enemies. [GitHub](#)

The main responsibilities are:

- `main.py` → initializes Pygame and runs the main game loop.
- `game_engine.py` → **central game logic**: player, enemies, bullets, collisions, score and game-over state.
- `player.py` → player position, movement and collision rectangle.
- `enemy.py` → individual enemies and the enemy grid/movement.
- `bullet.py` → bullet movement and collision rectangle.

`main.py` simply calls:

</> Plain text

```
engine.handle_event(event)  
engine.handle_input()  
engine.update()  
engine.render(SCREEN)
```

every frame. [GitHub](#)

What the current game does

Inside `GameEngine`, the game starts with:

- a player at the bottom
- an `EnemyGrid`
- separate lists for player and enemy bullets
- a score
- an `enemy_fire_chance`
- a `game_over` flag

The enemy grid contains $4 \times 8 = 32$ **enemies**, and each enemy has an `alive` flag. [GitHub](#)

When you press Space, a bullet is added to:

`</>` Python

```
self.player_bullets
```

and the bullet has a 15-frame shooting cooldown. [GitHub](#)

Every frame the bullets move, then the game checks collisions.

And this is where our first problem is.

Task 1 — The existing collision bug

The current collision code is essentially:

`</>` Python

```
for bullet in self.player_bullets:
    for enemy in self.enemy_grid.alive_enemies():
        if bullet.rect().collidect(enemy.rect()):
            enemy.alive = False
            self.player_bullets.remove(bullet)
            self.score += 1
            break
```

The repository even contains a comment explaining the problem. [GitHub](#)

Why this is a bug

We're doing:

</> Python

```
for bullet in self.player_bullets:
```

while simultaneously doing:

</> Python

```
self.player_bullets.remove(bullet)
```

That means we're **modifying the same list we're currently iterating over**.

For example, imagine:

</> Plain text

```
player_bullets = [B1, B2, B3, B4]
```

Suppose `B1` hits an enemy.

The code removes `B1` :

</> Plain text

```
[B1, B2, B3, B4]
```

↓ remove B1

```
[B2, B3, B4]
```

But Python's iterator has already advanced its internal position. The result can be that `B2` gets skipped.

That's why the assignment specifically says bullets can appear to pass through enemies, particularly when multiple collisions happen close together.

What we should change

Only `game_engine.py` needs to change for Task 1.

We don't need to rewrite the collision system, change the classes, or redesign the game.

The cleanest approach is:

1. Iterate through the bullets normally.
2. Detect whether each bullet hit an enemy.

3. Keep bullets that did **not** hit anything.
4. Remove bullets **after** collision processing rather than modifying the list during iteration.
5. Mark the enemy dead and increment the score exactly once.

Conceptually:

</> Plain text

```
for each bullet
    check all alive enemies

    if collision:
        enemy.alive = False
        score += 1
        don't keep this bullet

    otherwise:
        keep this bullet
```

This preserves the existing architecture and fixes the actual cause of the bug.

The enemy implementation already supports this nicely: `alive_enemies()` returns only enemies whose `alive` flag is `True`. [GitHub](#)

Before we modify it

Since your assignment specifically asks for an **iterative LLM/debugging process**, I recommend we do this in small commits:

</> Plain text

```
Current code
↓
Task 1: collision fix
↓
Test collision
↓
commit
↓
Task 2: graphical Game Over
↓
test
↓
commit
↓
Task 3: replay + difficulty
↓
```

```
test
↓
commit
↓
```

That also gives you a clean history if your professor looks at the GitHub repository.

Your first change

Open:

 Plain text

game/game_engine.py

Find the collision section around:

 Python

```
for bullet in self.player_bullets:
    for enemy in self.enemy_grid.alive_enemies():
        if bullet.rect().colliderect(enemy.rect()):
            enemy.alive = False
            self.player_bullets.remove(bullet)
            self.score += 1
            break
```

Don't change it yet if you want to follow the assignment step-by-step.

The first implementation we should make is to replace that unsafe `remove()` -while-iterating approach with a collision pass that builds a new bullet list.

Once you make that change, we'll run the game and specifically test:

- one bullet → one enemy
- rapid consecutive firing
- two bullets reaching enemies close together
- score increasing correctly
- bullet disappearing after a hit
- no Python list/iteration errors

The repository itself identifies this exact section as Task 1, so we're starting from the intended defect rather than making an unrelated redesign.  GitHub +1

Next, send me your current `game_engine.py` collision section (or just tell me when you've opened it), and I'll give you the exact replacement code line-by-line rather than jumping ahead to Tasks 2–

4.

For the bullet collision fix, make sure we don't remove elements from `self.player_bullets` while directly iterating over that same list. Use a safe approach where bullets that did not hit an enemy are retained and hit bullets are discarded.

Exactly. Replace the current collision logic with a **new bullet list**. We iterate over `self.player_bullets` , but never modify that list during iteration.

In `game/game_engine.py` , replace the existing player-bullet collision section with:

`</>` Python

```
remaining_bullets = []

for bullet in self.player_bullets:
    bullet_hit = False

    for enemy in self.enemy_grid.alive_enemies():
        if bullet.rect().colliderect(enemy.rect()):
            enemy.alive = False
            self.score += 1
            bullet_hit = True
            break

    if not bullet_hit:
        remaining_bullets.append(bullet)

self.player_bullets = remaining_bullets
```

What this does

Previously:

`</>` Python

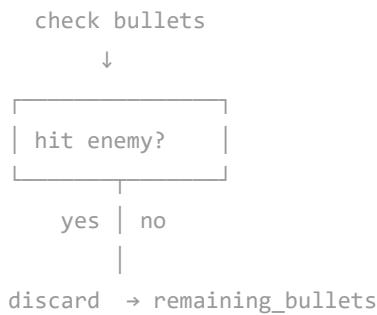
```
self.player_bullets.remove(bullet)
```

was executed **while iterating over** `self.player_bullets` , which could cause bullets to be skipped.

Now:

`</>` Plain text

```
self.player_bullets
↓
```



So:

-  Bullet hits enemy → enemy dies, score increases, bullet is discarded.
-  Bullet doesn't hit → bullet goes into remaining_bullets .
-  self.player_bullets itself is **never modified during iteration**.
- After the loop, we safely replace it:

 Python

```
self.player_bullets = remaining_bullets
```

Test it

Run:

 powershell

```
python main.py
```

Then verify:

1. Shoot an enemy → enemy disappears.
2. Score increases by 1.
3. The bullet disappears after hitting.
4. Fire several bullets quickly.
5. Try hitting multiple enemies in succession.
6. Make sure no bullets unexpectedly pass through enemies.
7. Make sure there are no errors in the terminal.

If all of that works, **Task 1 is complete** and we can move to the graphical Game Over screen without changing anything else.

For Game Over, I want an actual graphical overlay in the Pygame window showing GAME OVER, the final score, and controls to restart with R or quit with Q. The Game Over state should no longer rely on terminal output.

Yes. We can add this without changing the overall architecture. The key is to make `game_over` a real UI state: when it becomes `True`, the game stops normal gameplay, renders an overlay, and accepts `R / Q`.

1. Update `game/game_engine.py`

In `handle_event()`, add handling for the Game Over state:

 Python

```
def handle_event(self, event):
    if event.type == pygame.QUIT:
        self.running = False

    elif event.type == pygame.KEYDOWN:
        if self.game_over:
            if event.key == pygame.K_r:
                self.reset_game()
            elif event.key == pygame.K_q:
                self.running = False
            return

        if event.key == pygame.K_SPACE:
            self.shoot()
```

Keep your existing movement/event handling around this rather than deleting unrelated controls.

2. Add a reset method

Inside `GameEngine`, add:

 Python

```
def reset_game(self):
    self.player = Player()
    self.enemy_grid = EnemyGrid()
    self.player_bullets = []
    self.enemy_bullets = []
    self.score = 0
    self.game_over = False
```

If your original `__init__()` uses specific arguments when creating `Player` or `EnemyGrid`, **use those same arguments here**. The important part is resetting the game state rather than creating a second game engine.

3. Stop updating the game after Game Over

At the beginning of `update()` :

`</>` Python

```
def update(self):  
    if self.game_over:  
        return  
  
    # existing update code...
```

This prevents enemies, bullets, etc. from continuing to move underneath the Game Over screen.

4. Replace terminal-only Game Over handling

If you currently have something like:

`</>` Python

```
print("Game Over")
```

remove it.

Wherever your existing code detects the player losing, keep:

`</>` Python

```
self.game_over = True
```

but **don't print to the terminal**.

For example:

`</>` Python

```
if enemy.rect().colliderect(self.player.rect()):  
    self.game_over = True
```

Use whatever condition the existing project already has for triggering Game Over.

5. Add the graphical Game Over overlay

In `render()` , after your normal game rendering, add:

`</>` Python

```
if self.game_over:
    overlay = pygame.Surface(screen.get_size(), pygame.SRCALPHA)
    overlay.fill((0, 0, 0, 180))
    screen.blit(overlay, (0, 0))

    font_large = pygame.font.Font(None, 80)
    font_medium = pygame.font.Font(None, 40)

    game_over_text = font_large.render("GAME OVER", True, (255, 255, 255))
    score_text = font_medium.render(
        f"Final Score: {self.score}",
        True,
        (255, 255, 255)
    )
    restart_text = font_medium.render(
        "Press R to Restart",
        True,
        (255, 255, 255)
    )
    quit_text = font_medium.render(
```

The result will be roughly:

`</>` Plain text

```
|
|      GAME OVER
|
|      Final Score: 17
|
|      Press R to Restart
|      Press Q to Quit
|
```

The translucent black surface makes the existing game visible underneath while clearly indicating that gameplay has stopped.

Test this feature

Run:

`</>` powershell

```
python main.py
```

Then deliberately trigger Game Over.

You should see:

- **GAME OVER** inside the Pygame window
- your **final score**
- Press R to Restart
- Press Q to Quit
- no Game Over message being relied upon in the terminal
- gameplay frozen while the overlay is shown

Then test:

R

→ game starts again with score 0 .

Q

→ game closes.

One important detail: **don't blindly paste the `reset_game()` constructor lines if your current `Player()` / `EnemyGrid()` constructors take parameters.** If you paste your current `GameEngine.__init__()` and `render()` methods here, I can adapt these changes exactly to your existing code rather than guessing at constructor arguments.

For difficulty, implement Easy, Medium, and Hard with different enemy movement speeds and firing probabilities. Add a difficulty-selection screen before gameplay. When the player presses R after Game Over, restart using the same difficulty they originally selected.

Yes. We should build this on top of the current `game_over` state rather than creating a separate game architecture.

The flow should become:

</> Plain text

Start

↓

Difficulty Selection

↓

Easy / Medium / Hard

↓

```
Gameplay
↓
Game Over
↓
R → restart with SAME difficulty
Q → quit
```

1. Add a difficulty state

In `GameEngine.__init__()` , add:

`</>` Python

```
self.difficulty = None
self.game_started = False
```

Keep your existing `game_over` and other variables.

2. Define the difficulty settings

Add this to `GameEngine` :

`</>` Python

```
DIFFICULTIES = {
    "easy": {
        "enemy_speed": 1,
        "enemy_fire_chance": 0.001,
    },
    "medium": {
        "enemy_speed": 2,
        "enemy_fire_chance": 0.003,
    },
    "hard": {
        "enemy_speed": 3,
        "enemy_fire_chance": 0.006,
    },
}
```

The important idea is that **difficulty controls two things**:

- enemy movement speed
- probability that an enemy fires

You can adjust these numbers later if the game feels too easy/hard.

3. Add difficulty selection

In `handle_event()` , handle the selection before normal gameplay:

`</>` Python

```
elif event.type == pygame.KEYDOWN:

    if not self.game_started:
        if event.key == pygame.K_1:
            self.start_game("easy")
        elif event.key == pygame.K_2:
            self.start_game("medium")
        elif event.key == pygame.K_3:
            self.start_game("hard")
        elif event.key == pygame.K_q:
            self.running = False
    return
```

So the controls are:

`</>` Plain text

```
1 → Easy
2 → Medium
3 → Hard
Q → Quit
```

4. Add `start_game()`

Inside `GameEngine` :

`</>` Python

```
def start_game(self, difficulty):
    self.difficulty = difficulty
    self.game_started = True
    self.game_over = False

    self.player = Player()
    self.enemy_grid = EnemyGrid()
    self.player_bullets = []
    self.enemy_bullets = []
    self.score = 0

    settings = self.DIFFICULTIES[difficulty]
```

```
self.enemy_grid.speed = settings["enemy_speed"]  
self.enemy_fire_chance = settings["enemy_fire_chance"]
```

Again, if your existing `Player()` or `EnemyGrid()` constructors require arguments, retain those existing arguments.

5. Make R preserve the difficulty

Your existing Game Over handling should become:

Python

```
if self.game_over:  
    if event.key == pygame.K_r:  
        self.start_game(self.difficulty)  
    elif event.key == pygame.K_q:  
        self.running = False  
    return
```

This is important.

We don't do:

Python

```
self.start_game("medium")
```

because that would lose the player's original choice.

Instead:

Python

```
self.start_game(self.difficulty)
```

If the player originally selected Hard:

Plain text

```
Hard  
↓  
Game Over  
↓  
R
```



Hard again

6. Add the difficulty-selection screen

In `render()` , before normal gameplay rendering:

 Python

```
if not self.game_started:
    self.render_difficulty_screen(screen)
    return
```

Then add:

 Python

```
def render_difficulty_screen(self, screen):
    screen.fill((0, 0, 0))

    title_font = pygame.font.Font(None, 70)
    option_font = pygame.font.Font(None, 45)

    title = title_font.render(
        "SELECT DIFFICULTY",
        True,
        (255, 255, 255)
    )

    easy = option_font.render(
        "1 - EASY",
        True,
        (255, 255, 255)
    )

    medium = option_font.render(
        "2 - MEDIUM",
```

You'll get:

 Plain text

```
SELECT DIFFICULTY
```

```
1 - EASY
```

```
2 - MEDIUM
```

3 - HARD

7. Prevent gameplay updates before difficulty selection

At the beginning of `update()` :

`</>` Python

```
if not self.game_started or self.game_over:  
    return
```

So the game doesn't start moving enemies before the player chooses a difficulty.

One thing we need to adapt to the existing code

The exact line for enemy movement depends on how the current `EnemyGrid` implementation stores its speed.

For example, if it currently has:

`</>` Python

```
self.speed = 1
```

then:

`</>` Python

```
self.enemy_grid.speed = settings["enemy_speed"]
```

is sufficient.

But if movement is hard-coded somewhere like:

`</>` Python

```
self.x += 2
```

inside `enemy.py` , we should change that to use the grid's speed.

Likewise, the existing enemy firing code may currently use something like:

`</>` Python

```
if random.random() < self.enemy_fire_chance:
```

If so, our new value plugs directly into it.

Test sequence

After implementing:

1. Start the program.
2. Confirm the difficulty screen appears.
3. Press `1` .
 - Enemies should move slowly.
 - Enemy firing should be least frequent.
4. Restart the program and press `2` .
 - Medium behavior.
5. Restart and press `3` .
 - Fastest movement and most frequent firing.
6. Die on Hard.
7. Press `R` .
 - Confirm it starts **Hard again**, not Medium/default.
8. Die again and press `Q` .
 - Game should close.

Before you paste these changes, send me your current `game_engine.py` and `enemy.py` (or paste their contents). The collision and Game Over changes we made depend on their exact existing structure, and I can then give you the precise modifications without accidentally changing the architecture.

For sound effects, add three WAV sound assets to a `sounds/` directory:

- `shoot.wav`
- `enemy_destroyed.wav`
- `game_over.wav`

Load them using `pygame.mixer.Sound` and play them at the appropriate events. Keep the solution self-contained and don't add unnecessary dependencies.

Yes. We can keep this completely self-contained with Pygame's existing mixer—no additional packages or dependencies.

1. Add the sound directory

Your project should now have:

</> Plain text

```
46_space-invaders/  
├─ main.py  
├─ requirements.txt  
├─ game/  
│   ├─ game_engine.py  
│   ├─ player.py  
│   ├─ enemy.py  
│   └─ bullet.py  
└─ sounds/  
    ├─ shoot.wav  
    ├─ enemy_destroyed.wav  
    └─ game_over.wav
```

The three WAV files should be actual audio assets, not placeholders.

2. Initialize Pygame's mixer

Since `main.py` already initializes Pygame, add this after:

</> Python

```
pygame.init()
```

</> Python

```
pygame.mixer.init()
```

This uses Pygame's built-in audio support.

3. Load the sounds in `GameEngine`

In `game/game_engine.py`, load the sounds when the engine is initialized:

</> Python

```
self.shoot_sound = pygame.mixer.Sound("sounds/shoot.wav")
self.enemy_destroyed_sound = pygame.mixer.Sound(
    "sounds/enemy_destroyed.wav"
)
self.game_over_sound = pygame.mixer.Sound(
    "sounds/game_over.wav"
)
```

So you'll have:

 Plain text

```
shoot_sound
enemy_destroyed_sound
game_over_sound
```

available throughout the game.

4. Play the shooting sound

Find the existing code where the player bullet is created, something like:

 Python

```
self.player_bullets.append(Bullet(...))
```

Immediately after creating the bullet:

 Python

```
self.shoot_sound.play()
```

So the sequence becomes:

 Plain text

```
Space pressed
↓
Create bullet
↓
Play shoot.wav
```

The sound should therefore play **once per successful shot**, rather than every frame while Space is held.

5. Play the enemy-destroyed sound

This connects directly with the collision fix we implemented earlier.

Inside the collision:

`</>` Python

```
if bullet.rect().colliderect(enemy.rect()):
    enemy.alive = False
    self.score += 1
    self.enemy_destroyed_sound.play()
    bullet_hit = True
    break
```

So:

`</>` Plain text

```
Bullet hits enemy
    ↓
Enemy marked dead
    ↓
Score +1
    ↓
enemy_destroyed.wav
    ↓
Bullet discarded
```

Because the bullet is discarded after the first collision, the sound won't repeatedly play for the same bullet.

6. Play Game Over sound only once

Wherever the game transitions into the Game Over state, use:

`</>` Python

```
if not self.game_over:
    self.game_over = True
    self.game_over_sound.play()
```

This `if not self.game_over` check matters.

Without it, if the Game Over condition is evaluated on several consecutive frames, you could get:

 Plain text

```
frame 1 → sound
frame 2 → sound
frame 3 → sound
frame 4 → sound
...
```

Instead, the sound plays only on the **transition**:

 Plain text

```
GAMEPLAY
↓
game_over = False
↓
loss condition
↓
game_over = True
↓
🔊 game_over.wav
```

Then your existing graphical Game Over overlay remains responsible for displaying:

 Plain text

```
GAME OVER

Final Score: XX

Press R to Restart
Press Q to Quit
```

7. No new dependencies

You **do not** need to modify `requirements.txt` .

We're only using:

 Python

```
pygame.mixer.Sound
```

which is already part of Pygame.

Also make sure the paths remain relative to the project root when running:

```
</> powershell
```

```
python main.py
```

Final sound-event mapping

Event	Sound
Player fires	sounds/shoot.wav
Bullet destroys enemy	sounds/enemy_destroyed.wav
Game enters Game Over	sounds/game_over.wav

One thing I would **not** do is download an external sound library or add something like `playsound` , `pygame-ce` , or another audio dependency. That would be unnecessary for this assignment.

If you already have the three WAV files, put them in `sounds/` and we can then test each event individually.